

JAVA RMI ASYNCHRONOUS CALL

Info408

Houssein WEHBE

Problem of the Synchronous Model

- RMI is a remote method call mechanism, which by default corresponds to a synchronous model:
 - the client waits for the server to send it a response.
- This model works perfectly but is very restrictive, especially in situations where processing on the server can be relatively long.
- We will study in this section different techniques which allow to build an asynchronous model above RMI.

Plan

1. Revision : Threads and synchronization
2. Asynchronous RMI call

Ex 1 : Threads Creation

- Write a Java program containing two threads (in addition to the main thread) so each displays the list of integers between 1 and n (n is a parameter of their constructor). We can introduce a random pause between two displays to properly view the ordering between the threads

Ex 1

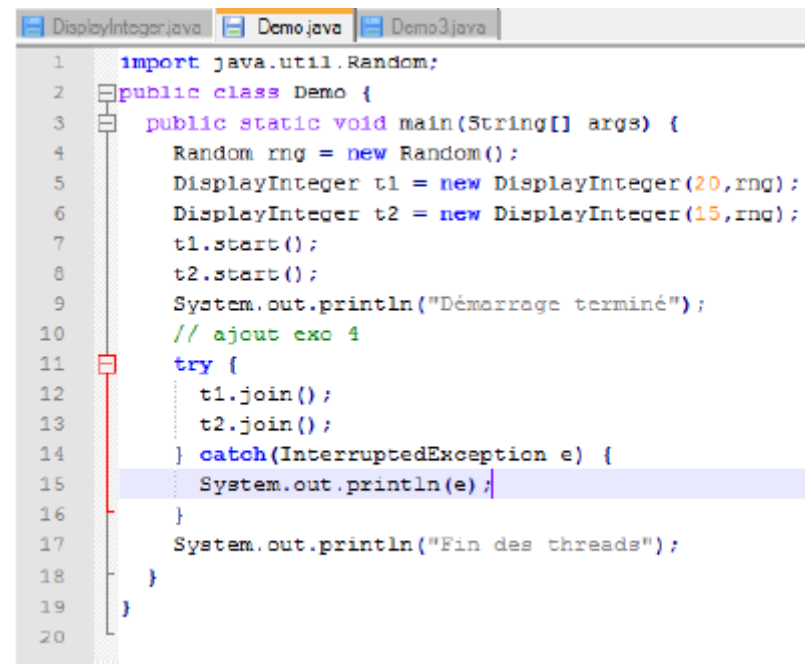
```
1  import java.util.Random;
2  public class DisplayInteger extends Thread {
3      private int max;
4      private Random rng;
5      public DisplayInteger(int n, Random rng) {
6          max = n;
7          this.rng = rng;
8      }
9      public void run() {
10         String name = getName();
11         for(int i=1; i<=max; i++) {
12             System.out.println(name+" "+i);
13             try {
14                 Thread.sleep(Math.abs(rng.nextInt())%1500);
15             } catch (InterruptedException e) {
16             }
17         }
18         System.out.println(name+" terminé");
19     }
20 }
21
```

DisplayInteger.java Demo.java Demo3.java

```
1  import java.util.Random;
2  public class Demo {
3      public static void main(String[] args) {
4          Random rng = new Random();
5          DisplayInteger t1 = new DisplayInteger(20, rng);
6          DisplayInteger t2 = new DisplayInteger(15, rng);
7          t1.start();
8          t2.start();
9          System.out.println("Démarrage terminé");
10     }
11 }
12
```

Ex 2 : Basic synchronization using Join

- Modify the program of the previous exercise to display a message to the main method when the two threads are finished



```
1  import java.util.Random;
2  public class Demo {
3      public static void main(String[] args) {
4          Random rng = new Random();
5          DisplayInteger t1 = new DisplayInteger(20,rng);
6          DisplayInteger t2 = new DisplayInteger(15,rng);
7          t1.start();
8          t2.start();
9          System.out.println("Démarrage terminé");
10         // ajout exo 4
11         try {
12             t1.join();
13             t2.join();
14         } catch(InterruptedException e) {
15             System.out.println(e);
16         }
17         System.out.println("Fin des threads");
18     }
19 }
20
```

Ex 3: Busy Waiting

- Modify the thread of the previous exercise to incorporate the following functionalities:
 - *a private variable of the active thread indicates whether or not it should run.*
 - This variable is initialized to true (the thread turns) by the constructor.
 - The class proposes a method allowing to suspend the thread and another to restart it, by changing the value of active;
- Write a program which demonstrates the suspension mechanism thus obtained (with at least two threads in addition to the main thread).

```

1  import java.util.Random;
2  public class DisplayInteger extends Thread {
3      private int max;
4      private Random rng;
5      private boolean actif;
6      public DisplayInteger(int n, Random rng) {
7          max = n;
8          this.rng = rng;
9          actif = true;
10     }
11     public void run() {
12         String name = getName();
13         for(int i=1; i<=max; i++) {
14             while(!actif) {
15                 // attente active
16                 System.out.println("attente active (" + name + ")");
17                 try {
18                     Thread.sleep(1000);
19                 } catch (InterruptedException e) {
20                 }
21             }
22             System.out.println(name + ": " + i);
23             try {
24                 Thread.sleep(Math.abs(rng.nextInt()) % 1500);
25             } catch (InterruptedException e) {
26             }
27         }
28         System.out.println(name + " terminé");
29     }
30     ...

```

```

1  import java.util.Random;
2  public class Demo {
3      public static void main(String[] args) {
4          Random rng = new Random();
5          DisplayInteger t1 = new DisplayInteger(20, rng);
6          DisplayInteger t2 = new DisplayInteger(15, rng);
7          t1.start();
8          t2.start();
9          System.out.println("Démarrage terminé");
10         t1.suspend();
11         try {
12             Thread.sleep(4000);
13         } catch (InterruptedException e) {
14         }
15         t2.suspend();
16         t1.reprendre();
17         try {
18             Thread.sleep(4000);
19         } catch (InterruptedException e) {
20         }
21         t2.reprendre();
22     }
23 }
24
25
26
27
28
29
30 public void suspendre() {
31     actif = false;
32     System.out.println("suspension de " + getName());
33 }
34 public void reprendre() {
35     actif = true;
36     System.out.println("reprise de " + getName());
37 }
38
39

```


Ex 4: Using of Synchronized, Wait, Notify, NotifyAll

- Modify the thread of the previous exercise in order to **replace the active wait with a passive waiting mechanism based on wait and notify**. We will proceed as follows:
 - The active wait will be replaced by a synchronized block in which the pause is replaced by a wait;
 - the thread control methods will be declared synchronized and the restart method will use notify to wake up the thread.
- Reuse the demonstration program to test the suspension mechanism thus obtained

```

public class DisplayInteger extends Thread {
    private int max;
    private Random rng;
    private boolean actif;
    public DisplayInteger(int n, Random rng) {
        max = n;
        this.rng = rng;
        actif = true;
    }
    public void run() {
        String name = getName();
        for(int i=1; i<=max; i++) {
            synchronized(this) {
                while(!actif) {
                    // attente passive
                    System.out.println("attente passive (" + name + ")");
                    try {
                        wait();
                    } catch (InterruptedException e) {
                    }
                }
                System.out.println(name + ": " + i);
                try {
                    Thread.sleep(Math.abs(rng.nextInt()) * 1500);
                } catch (InterruptedException e) {
                }
            }
            System.out.println(name + " terminé");
        }
    }
}

```

```

32 public synchronized void suspendre() {
33     actif = false;
34     System.out.println("suspension de " + getName());
35 }
36 public synchronized void reprendre() {
37     actif = true;
38     System.out.println("reprise de " + getName());
39     notify();
40 }
41 }
42

```

```

DisplayInteger.java Demo.java Demo3.java DisplayInteger.java Demo.java
1 import java.util.Random;
2 public class Demo {
3     public static void main(String[] args) {
4         Random rng = new Random();
5         DisplayInteger t1 = new DisplayInteger(20, rng);
6         DisplayInteger t2 = new DisplayInteger(15, rng);
7         t1.start();
8         t2.start();
9         System.out.println("Démarrage terminé");
10        t1.suspendre();
11        try {
12            Thread.sleep(4000);
13        } catch (InterruptedException e) {
14        }
15        t2.suspendre();
16        t1.reprendre();
17        try {
18            Thread.sleep(4000);
19        } catch (InterruptedException e) {
20        }
21        t2.reprendre();
22    }
23 }
24

```

Plan

1. Revision : Threads and synchronization
2. Asynchronous RMI call

Ex5: Asynchronous RMI call

- To make an asynchronous RMI call at the client level, you can **encapsulate the call in a thread** as illustrated in the following exercise:
- **Write an RMI server** which offers a classic echo method (it sends the call parameter character string to the client), but with a significant pause at the start of the method, to simulate a complex calculation (we can randomly select the length of the break).
- **Write a client** for this server which makes the call asynchronously by doing the following:
 - write a **Thread (or Runnable) object which makes the call and then displays the result;**
 - start the call in the main thread and execute in parallel (always in this main thread) any operations (such as for example some displays interspersed with pauses).

```

1 import java.rmi.*;
2 public interface IServeur extends Remote {
3     public String echo(String msg) throws RemoteException;
4 }
5

```

```

1 import java.rmi.*;
2 import java.rmi.server.*;
3 import java.util.*;
4 public class Serveur extends UnicastRemoteObject implements IServeur {
5     public Serveur() throws RemoteException {
6         super();
7     }
8     public String echo(String msg) throws RemoteException {
9         System.out.println("Ping");
10        try {
11            Thread.sleep(5000);
12        } catch (InterruptedException e) {
13        }
14        System.out.println("Pong");
15        return "["+msg+"]";
16    }
17    public static void main(String[] args) throws Exception {
18        // démarrage du serveur
19        Serveur serveur = new Serveur();
20        Naming.rebind("ServeurEcho", serveur);
21        System.out.println("Serveur démarré");
22    }
23 }
24

```

```

1 import java.rmi.*;
2 public class Client {
3     public static void main(String[] args) throws Exception {
4         // le final est ajouté à cause de la classe anonyme
5         final IServeur serveur = (IServeur) Naming.lookup("ServeurEcho");
6         // solution avec une classe anonyme
7         Thread t = new Thread(new Runnable() {
8             public void run() {
9                 try {
10                     System.out.println(serveur.echo("coucou"));
11                 } catch (RemoteException e) {
12                     System.out.println(e);
13                 }
14             }
15         });
16        t.start();
17        System.out.println("démarrage de l'appel");
18        for(int i=1; i<=5; i++) {
19            System.out.println(i);
20            try {
21                Thread.sleep(1500);
22            } catch (InterruptedException e) {
23            }
24        }
25        System.out.println("fin du main");
26    }
27 }
28

```

The problem with this exercise is that it is the thread that executes the call that decides to display the result

Ex6: Using of wait/notify in client side

- Modify the asynchronous call object of the previous exercise to obtain the following functionalities:
 - the run **method should no longer display the result** but simply store it in an appropriate variable;
 - *the object must propose a method allowing to know if the computation is finished;*
 - **the object must propose a blocking method returning the result of the call** (the blocking will be based on the wait / notify pair)
- Provide a demonstration of the functionality of this new asynchronous call

```

public class AppelAsynchrone extends Thread {
    private IServeur serveur;
    private boolean done;
    private String result;
    private String param;
    public AppelAsynchrone(IServeur serveur,String param) {
        this.serveur = serveur;
        done = false;
        this.param = param;
    }
    public void run() {
        try {
            result = serveur.echo(param);
        } catch (RemoteException e) {
            System.out.println(e);
        }
        synchronized(this) {
            done = true;
            notify();
        }
    }
    public synchronized boolean isDone() {
        return done;
    }
    public synchronized String getResult() {
        while(!done) {
            try {
                wait();
            } catch (InterruptedException e) {
            }
        }
        return result;
    }
}

```

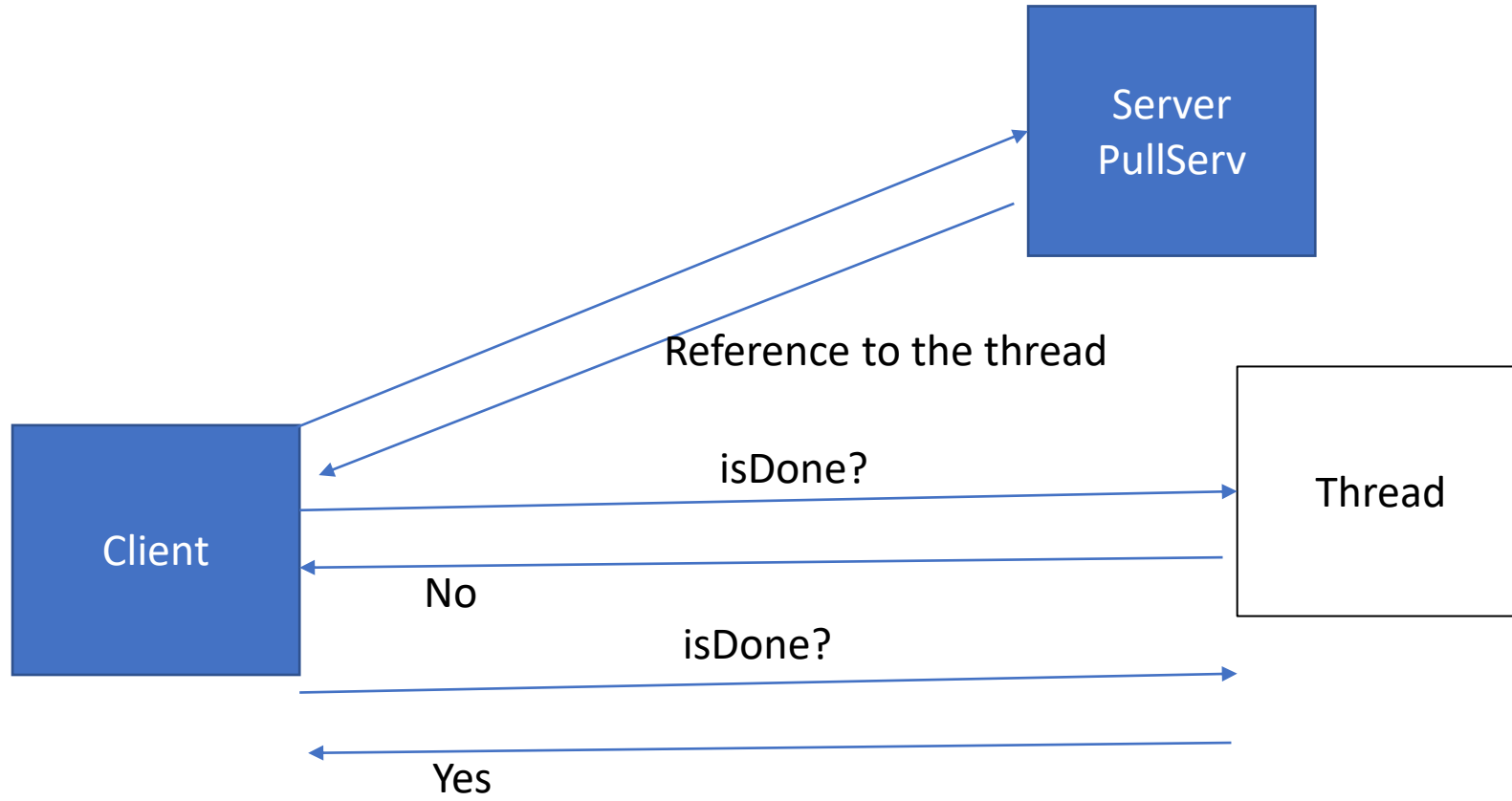
```

1 import java.rmi.*;
2 public class Client {
3     public static void main(String[] args) throws Exception {
4         IServeur serveur = (IServeur)Naming.lookup("ServeurEcho");
5         AppelAsynchrone appel = new AppelAsynchrone(serveur,"coucou");
6         appel.start();
7         System.out.println("démarrage de l'appel");
8         for(int i=1;i<=5;i++) {
9             System.out.println(i);
10            try {
11                Thread.sleep(500);
12            } catch (InterruptedException e) {
13            }
14        }
15        System.out.println("Attente du résultat");
16        System.out.println(appel.getResult());
17        System.out.println("fin du main");
18    }
19 }

```

Ex7: Pulling solution

- If the server offers services allowing asynchronous calls, client programming is simplified.
- We start the server part with a pulling API:
 - the client initiates an operation on the server,
 - then periodically asks the server if the operation is finished.
 - The client is therefore active and generally performs an active wait



Ex7: Pulling solution

```
import java.rmi.*;

public interface IPullServer extends Remote {
    public IDoSomethingResult doSomething(String param) throws RemoteException;
}
```

- The doSomething method is **asynchronous**, that is to say it must launch a server-side calculation (using a thread) and return control to the client as quickly as possible.
- To allow the client to obtain the result of the operation, the method returns a reference to a remote IDoSomethingResult object specific to the call.

```
import java.rmi.*;

public interface IDoSomethingResult extends Remote {
    public boolean isDone() throws RemoteException;
    public String getResult() throws RemoteException;
}
```

Ex7: Pulling solution

1- Implement IDoSomethingResult by a class which also *implements Runnable* (and which inherits from UnicastRemoteObject): the idea is to have the code to be executed (the run method) with the storage parameter (for the isDone and getResult methods) and the RMI server (thanks to UnicastRemoteObject).

- For doSomething calculations, we will be satisfied with an echo with a random response time.

2. Implement IPullServer.

3. Implement a client using IPullServer to make an asynchronous call with an active wait

```

1 import java.rmi.*;
2 public interface IPullServer extends Remote {
3     public IDoSomethingResult doSomething(String param) throws RemoteException;
4 }
5

```

```

1 import java.rmi.*;
2 public interface IDoSomethingResult extends Remote {
3     public boolean isDone() throws RemoteException;
4     public String getResult() throws RemoteException;
5 }
6

```

```

1 import java.rmi.*;
2 import java.rmi.server.*;
3 public class PullServer extends UnicastRemoteObject implements IPullServer {
4     public PullServer() throws RemoteException {
5         super();
6     }
7     public IDoSomethingResult doSomething(String param) throws RemoteException {
8         System.out.println("Ping");
9         DoSomethingResult result = new DoSomethingResult(param);
10        Thread support = new Thread(result);
11        support.start();
12        System.out.println("Pong");
13        return result;
14    }
15    public static void main(String[] args) throws Exception {
16        // démarrage du serveur
17        PullServer serveur = new PullServer();
18        Naming.rebind("ServerPull", serveur);
19        System.out.println("Serveur démarré");
20    }
21 }
22
23

```

```

import java.rmi.*;
import java.rmi.server.*;
public class DoSomethingResult extends UnicastRemoteObject
implements IDoSomethingResult, Runnable {
    private String param;
    private String result;
    private boolean done;
    public DoSomethingResult(String param) throws RemoteException {
        super();
        this.param = param;
        done = false;
    }
    public boolean isDone() throws RemoteException {
        System.out.println("isDone");
        return done;
    }
    public String getResult() throws RemoteException {
        System.out.println("getResult");
        return result;
    }
    public void run() {
        // le calcul proprement dit
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
        }
        result = "[" + param + "]";
        done = true;
    }
}

```

Ex7: Pulling solution

```
import java.rmi.*;
public class Client {
    public static void main(String[] args) throws Exception {
        IPullServer serveur = (IPullServer)Naming.lookup("ServerPull");
        System.out.println("démarrage de l'appel");
        IDoSomethingResult result = serveur.doSomething("coucou");
        boolean first = true;
        for(int i=1;i<=5;i++) {
            System.out.println(i);
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
            }
            System.out.println("Attente du résultat");
            if(first && result.isDone()) {
                System.out.print("Le voilà : ");
                System.out.println(result.getResult());
                first = false;
            }
        }
        System.out.println("fin du main");
    }
}
```

Ex8: Pulling solution with a passive wait

```
public void waitForResult() throws RemoteException;
```

- Add this method to the IDoSomethingResult interface
- This method allows the client to avoid active waiting: **it blocks the client until the result is available.**
- *Implement this new version by using the wait / notify couple to implement blocking on the server*

```

import java.rmi.*;

public interface IDoSomethingResult extends Remote {
    public boolean isDone() throws RemoteException;
    public String getResult() throws RemoteException;
    public void waitForResult() throws RemoteException;
}

public class DoSomethingResult extends UnicastRemoteObject
implements IDoSomethingResult, Runnable {
    private String param;
    private String result;
    private boolean done;

    public DoSomethingResult(String param) throws RemoteException {
        super();
        this.param = param;
        done = false;
    }

    public synchronized boolean isDone() throws RemoteException {
        System.out.println("isDone");
        return done;
    }

    public synchronized String getResult() throws RemoteException {
        System.out.println("getResult");
        return result;
    }

    public synchronized void waitForResult() throws RemoteException {
        System.out.println("waitForResult");
        while(!done) {
            try {
                wait();
            } catch (InterruptedException e) {
            }
        }
    }

    public void run() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
        }

        synchronized(this) {
            result = "[" + param + "]";
            done = true;
            notify();
        }
    }
}

```

```

import java.rmi.*;

public interface IPullServer extends Remote {
    public IDoSomethingResult doSomething(String param) throws RemoteException;
}

import java.rmi.*;
import java.rmi.server.*;

public class PullServer extends UnicastRemoteObject implements IPullServer {
    public PullServer() throws RemoteException {
        super();
    }

    public IDoSomethingResult doSomething(String param) throws RemoteException {
        System.out.println("Ping");
        DoSomethingResult result = new DoSomethingResult(param);
        Thread support = new Thread(result);
        support.start();
        System.out.println("Pong");
        return result;
    }

    public static void main(String[] args) throws Exception {
        // démarrage du serveur
        PullServer serveur = new PullServer();
        Naming.rebind("ServerPull", serveur);
        System.out.println("Serveur démarré");
    }
}

```

Ex8: Pulling solution with a passive wait

```
import java.rmi.*;
public class Client {
    public static void main(String[] args) throws Exception {
        IPullServer serveur = (IPullServer)Naming.lookup("ServerPull");
        System.out.println("démarrage de l'appel");
        IDoSomethingResult result = serveur.doSomething("coucou");
        boolean first = true;
        for(int i=1;i<=5;i++) {
            System.out.println(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
            }
            System.out.println("Attente du résultat");
            if(first && result.isDone()) {
                System.out.print("Le voilà : ");
                System.out.println(result.getResult());
                first = false;
            }
        }
        if(first) {
            System.out.println("Toujours pas de résultat, on passe en attente passive");
            result.waitForResult();
            System.out.print("Le voilà : ");
            System.out.println(result.getResult());
        }
        System.out.println("fin du main");
    }
}
```


Ex9: Callback solution (push)

- the client registers with the server and **the server calls the client to inform them of the availability of the result.**
- The client only needs to create a server object (a callback) which is transmitted (in the form of a remote reference) to the server itself during registration.
- When the server wants to notify the client, it calls a method remote from the callback object.
- To simplify the deployment, we can use the decorator design pattern for the implementation of the callback.

IPushServer

```
import java.rmi.*;
public interface IPushServer extends Remote {
    public void doSomething(String param, IDoSomethingCallback callback)
        throws RemoteException;
}
```

- **The doSomething method is asynchronous:** it launches the server-side calculation and returns control as quickly as possible.
- The IDoSomethingCallback remote reference corresponds to an object created on the client that the server will use to inform the client of the completion of the operation. The interface is very simple

IDoSomethingCallback

```
import java.rmi.*;
public interface IDoSomethingCallback extends Remote {
    public void reportCompletion(String result) throws RemoteException;
}
```

Ex9: CallBack solution (push)

- 1- Implement IPushServer: the operation is a simple echo with a random response time.
- 2- Implement IDoSomethingCallback in the simplest way possible, for example by making a display (on the client) by the reportCompletion method.
3. Implement a client to test the mechanism.
4. Test the behavior of the server when the client finishes waiting for the call to be completed

```

import java.rmi.*;

public interface IPushServer extends Remote {
    public void doSomething(String param, IDoSomethingCallback callback)
        throws RemoteException;
}

import java.rmi.*;
import java.rmi.server.*;

public class PushServer extends UnicastRemoteObject implements IPushServer {
    public PushServer() throws RemoteException {
        super();
    }
    public void doSomething(String param, IDoSomethingCallback callback)
        throws RemoteException {
        System.out.println("Ping");
        DoSomething support = new DoSomething(param, callback);
        support.start();
        System.out.println("Pong");
    }
    public static void main(String[] args) throws Exception {
        // démarrage du serveur
        PushServer serveur = new PushServer();
        Naming.rebind("ServerPush", serveur);
        System.out.println("Serveur démarré");
    }
}

```

```

import java.rmi.*;

public class DoSomething extends Thread {
    private String param;
    private IDoSomethingCallback callback;
    public DoSomething(String param, IDoSomethingCallback callback) {
        this.param = param;
        this.callback = callback;
    }
    public void run() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
        }
        try {
            callback.reportCompletion(param);
        } catch (RemoteException e) {
            System.err.println("Cannot report completion: "+e);
        }
        callback = null;
        System.gc();
    }
}

```

Client

```
import java.rmi.*;

public interface IDoSomethingCallback extends Remote {
    public void reportCompletion(String result) throws RemoteException;
}

import java.rmi.*;
import java.rmi.server.*;
public class DoSomethingCallback extends UnicastRemoteObject
    implements IDoSomethingCallback {
    public DoSomethingCallback() throws RemoteException {
        super();
    }
    public void reportCompletion(String result) throws RemoteException {
        System.out.println(result);
    }
}
```

```
1 import java.rmi.*;
2 public class Client {
3     public static void main(String[] args) throws Exception {
4         // création de la callback
5         DoSomethingCallback callback = new DoSomethingCallback();
6         IPushServer serveur = (IPushServer)Naming.lookup("ServerPush");
7         System.out.println("démarrage de l'appel");
8         serveur.doSomething("coucou", callback);
9         for(int i=1; i<=5; i++) {
10             System.out.println(i);
11             try {
12                 Thread.sleep(1500);
13             } catch (InterruptedException e) {
14             }
15         }
16         System.out.println("fin du main");
17     }
18 }
```

Ex10:

- Add to the implementation of `IDoSomethingCallback` built in the previous exercise, methods inspired by the previous exercises allowing the client to actively and/or passively wait for the server call. Modify the test program to demonstrate the new features.
- (Main of the client will periodically ask the callback object if the results is done, you can let it wait if the results are not yet received)

```

public class Client {
    public static void main(String[] args) throws Exception {
        // création de la callback
        DoSomethingCallback callback = new DoSomethingCallback();
        IPushServer serveur = (IPushServer)Naming.lookup("ServerPush");
        System.out.println("démarrage de l'appel");
        serveur.doSomething("coucou",callback);
        boolean first = true;
        for(int i=1;i<=5;i++) {
            System.out.println(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
            }
            System.out.println("Attente du résultat");
            if(first && callback.isDone()) {
                System.out.print("Le voilà : ");
                System.out.println(callback.getResult());
                first = false;
            }
        }
        if(first) {
            System.out.println("Toujours pas de résultat, on passe en attente passive");
            callback.waitForResult();
            System.out.print("Le voilà : ");
            System.out.println(callback.getResult());
        }
        System.out.println("fin du main");
    }
}

```